Mule Account Detection via Gradient Boosting

A Feature-Engineered XGBoost + SMOTE Pipeline for Severe Class Imbalance

Technical Whitepaper — PSB's Cybershield Hackathon by IIT Hyderabad BOI, June 2026

1. Abstract

This paper presents a machine learning pipeline for automated mule account detection applied to a dataset of 9,082 bank accounts with 3,924 features and a severe class imbalance of 0.89% (81 positive instances). The pipeline integrates systematic data leakage removal, Random Forest importance-based feature selection to a ~51-feature subset, and a final XGBoost classifier trained with SMOTE oversampling at a tuned decision threshold of 0.45. On the held-out test set, the system achieves 81.2% mule recall with a 0.22% false alarm rate, equating to 1.3 investigation alerts per confirmed mule — a viable operational configuration for a bank fraud workflow. Out-of-fold (OOF) cross-validation over all 65 training mules yields ROC-AUC of 0.9680 and mule F1 of 0.76, providing a conservative and reliable generalization estimate beyond the 16-mule test set. SHAP (TreeExplainer) identifies banking relationship depth (F3898) as the dominant predictor, with mean absolute SHAP value approximately 2.0 — nearly 3x that of the second-ranked feature.

2. Problem Statement & Dataset

Mule accounts function as intermediary nodes in fraud chains: stolen funds are routed through multiple accounts before reaching the fraudster, obfuscating the money trail across transfer hops. Traditional rule-based monitoring systems — flag if more than N transactions per hour, flag if transfer exceeds threshold T — are trivially circumvented by adversaries who learn the rules and operate just beneath them. The objective is to learn latent behavioural and transactional patterns from historical account-level data to classify prospective accounts as mule or legitimate.

Dataset Parameter	Value
Total accounts	9,082
Feature count	3,924 (F0 through F3923)
Target variable	F3924 (1 = mule, 0 = legitimate)
Mule accounts	81 (0.89% positive class rate)
Legitimate accounts	9,001
Class ratio (neg : pos)	111 : 1
Hint features	18 candidate predictors provided by the problem statement

The 111:1 imbalance has direct consequences for evaluation strategy. Predicting all-legitimate achieves 99.1% accuracy while catching zero fraud — accuracy is therefore uninformative. All model comparisons use ROC-AUC as the primary metric: it evaluates discriminative ranking across every possible threshold,

answering the question of whether a randomly selected mule receives a higher suspicion score than a randomly selected legitimate account, independent of any chosen operating point.

3. Data Cleaning & EDA

3.1 Null Handling, Variance Filtering, and Imputation

Features exceeding 50% missing values were dropped. A column with greater than 50% missing entries encodes data recording behaviour rather than account behaviour — any pattern learned from it is a pattern in which accounts had measurements taken, not a pattern in the accounts themselves. Near-zero variance columns were also removed: they offer no discriminative power and add noise to subsequent models. Remaining missing values were imputed with column medians rather than means. Financial distributions are heavily right-skewed; the median is insensitive to the extreme outliers (e.g., an account transacting fifty crore rupees against a modal ten thousand) that substantially displace the mean, making mean imputation systematically biased toward unrepresentative values.

3.2 Data Leakage: F2230

The most significant finding in the cleaning phase was F2230, a month-year code feature. Distributional inspection revealed that all 9,001 legitimate accounts shared the single value "Oct25", while the 81 mule accounts were distributed across earlier months. A model retaining this feature would learn the rule F2230 ≠ Oct25 → mule and score near-perfectly on historical data. In production, any incoming account carries the current calendar month — the feature carries no generalizable signal and collapses entirely on prospective data. F2230 was removed before any modeling.

This is a canonical data leakage case: information present in training data that would not exist, or would not carry the same discriminative signal, for prospective accounts. The diagnostic heuristic: any feature with a suspiciously strong univariate association with the target demands a mechanistic explanation. If no causal story can be told without circular reference to the label itself, the feature is suspect regardless of the apparent performance gain its inclusion provides.

3.3 Categorical Encoding

Categorical features were encoded with semantic intent. F3889, encoding account age in ordered buckets (L7D, L14D, L31D, L60D, L90D, L180D, G365D), was mapped to integers 1 through 7 preserving chronological order. The ordinal relationship is semantically real: an account aged more than 365 days is meaningfully different from one aged 7 days, and this ordering must be preserved. Nominal features with no inherent ordering were label-encoded. For tree-based models this distinction is less critical (trees split on equality conditions, not magnitude), but maintaining it prevents subtle errors if the feature set is later passed to a gradient- or distance-based model.

3.4 EDA and Signal Analysis

Standard histogram-based distribution comparisons were ineffective: the extreme right skew of financial data compresses both class distributions into an indistinguishable left margin. A class-conditional mean

ratio table (mule mean / legitimate mean per feature) proved more diagnostic. Features with ratios substantially diverging from 1.0 were identified as signal candidates; ratios within ±5% of 1.0 were deprioritized. For features with extreme outliers, both mean and median ratios were cross-checked — F3836 illustrates the risk of relying on a single statistic: a legitimate account mean of −49M (driven by outliers) vs. a mule mean of 338K appeared to show extreme divergence; the medians (382K vs. 338K) revealed near-identical distributions.

Feature	Signal Direction	Approx. Ratio (Mule/Legit)	Assessment
F2678	Mules lower	0.03	Strongest signal — mules 97% lower on average
F2122	Mules lower	0.11	Very clean separation; 9x difference
F2956	Mules lower	~0.50	Reliable moderate signal
F2737	Mules lower	~0.45	Reliable moderate signal
F1692	Mules lower	~0.43	Reliable moderate signal
F670	Mules higher	2.60	Only hint feature where mules are elevated
F3836	Weak	Mean misleading	Medians near-identical; outlier-driven mean divergence

4. Feature Engineering & Selection

4.1 Engineering Strategy

Feature engineering translates domain hypotheses into model-readable representations. A tree-based model can in principle discover cross-feature interactions by chaining splits, but pre-computing those relationships reduces the required tree depth and makes signal more accessible. Four strategies were applied, in order of effectiveness on this dataset:

Strategy	Example	Rationale
Ratio features	F670 / (F2678 + 1e-6)	Amplifies divergence where one feature is elevated for mules and another depressed. The 1e-6 denominator guard prevents division-by-zero, replacing infinity with a large but finite value.
Interaction products	F2678 × F2122	Both features are lower for mules; their product is multiplicatively smaller: a 126x difference on one axis and a 9x difference on the other yields ~1,159x separation in the product column — a far stronger split signal.
Composite aggregate (sum_of_lows)	F2678 + F2122 + F2956 + F2737 + F1692	All five components are consistently lower for mules. The sum is a composite activity score: a mule account low on all five produces a very small sum; a legitimate account low on one or two still accumulates a moderate sum from the others, making the composite more robust to per-feature noise.
Z-score features	(F_i − mean) / std	Largely ineffective: with 9,001 legitimate accounts, global mean and std are dominated by the majority class. Mule z-scores converge toward the legitimate median. Z-scores work best when both classes are reasonably balanced or when the minority is extreme in the tails.

Engineered features were validated before selection using three diagnostic lenses: median ratio (reliable for dense features); mean ratio (informative for sparse features where median collapses to zero); and zero-percentage gap (captures sparsity pattern differences that both median and mean obscure). A feature was retained if any single lens exceeded its signal threshold — OR logic prevents systematically discarding sparse features that only show signal through their sparsity pattern.

4.2 RF-Based Feature Selection

A Random Forest (500 trees, min_samples_leaf=3, min_samples_split=6, class_weight='balanced', max_features='sqrt') was trained on all 2,000+ post-engineering features to rank them by Gini importance. Hyperparameter choices were driven by the 65-sample training minority class: min_samples_leaf=3 compels every leaf to represent at least three accounts, forcing generalisation rather than memorisation of individual mule samples. class_weight='balanced' upweights each mule ~111x to prevent the RF from defaulting to all-legitimate predictions.

The top 50 features by importance were selected. sum_of_lows was force-included despite ranking below position 50: diagnostic validation confirmed genuine signal (median ratio 0.648), and the RF importance metric can understate composite features when their component features are also present. The F38xx range dominated the final set — F3898 in particular showed a strong class-conditional distribution: 53% of legitimate accounts had value 3 while only 2.5% of mules did; 48% of mules had value 0 versus 31% of legitimate accounts. This is consistent with banking relationship depth — mule accounts, created and immediately exploited, lack the history to accumulate relationship indicators. The final feature set contained 51 features.

5. Baseline Models

5.1 Architecture and Split

Data was partitioned 80/20 (train/test) with stratify=y to guarantee proportional mule representation in both splits. Without stratification, the 81-mule dataset can produce highly variable minority class allocations: the difference of one mule in the test set shifts all recall metrics by 6.25 percentage points. All preprocessing was embedded in a sklearn Pipeline. This is not merely code hygiene — it ensures that statistics computed during preprocessing (StandardScaler mean and variance) are fit exclusively on the training partition and applied to the test partition without refitting, closing the door on a specific class of test-set contamination.

5.2 Model Configurations

Model	Key Configuration	Rationale
Logistic Regression	class_weight='balanced'; StandardScaler in Pipeline	Establishes a linear floor. class_weight='balanced' penalises mule misclassifications ~111x more heavily. StandardScaler required: LR optimises a gradient-based objective sensitive to feature magnitude. Anything that cannot beat LR is not worth deploying.
Random Forest	700 trees; class_weight='balanced_subsample'; min_samples_leaf=3; no scaling	balanced_subsample recomputes class weights per bootstrap sample, creating additional diversity across trees on imbalanced data. min_samples_leaf=3 prevents memorization of individual mule accounts. Trees split on threshold inequalities — scaling is irrelevant.

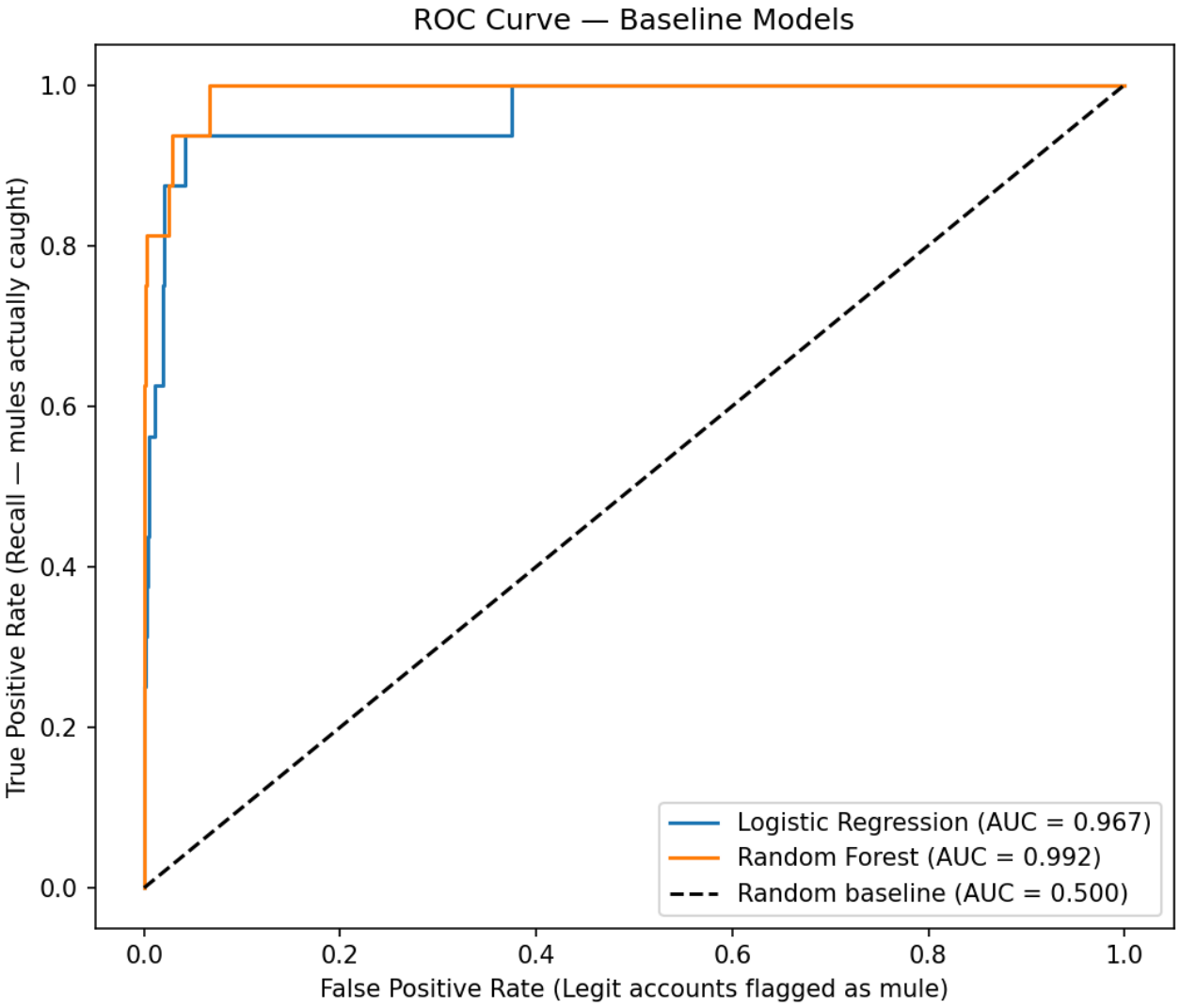
5.3 Results and Interpretation

Model	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.0505	0.9375	0.0958	0.9672
Random Forest	0.9000	0.5625	0.6923	0.9918

These results do not show LR outperforming RF — they show fundamentally different operating points on the same precision-recall curve. LR flagged approximately 297 accounts: 15 of 16 test mules caught, but 282 false alarms (95% false alarm rate). That is operationally unacceptable regardless of recall: investigating 297 accounts to find 15 mules means 95% of analyst effort is wasted. RF flagged 10 accounts: 9 of 16 mules caught, only 1 false alarm (90% precision). ROC-AUC is the correct comparison metric — it evaluates discriminative ranking across all thresholds simultaneously, independent of the default 0.50 cutoff. RF's 0.9918 vs. LR's 0.9672 establishes RF as the superior model. RF's conservative behavior at threshold 0.50 is a threshold choice, not a model quality limitation.

The general principle: on imbalanced classification problems, the goal is not maximum recall. It is maximum recall subject to precision being operationally viable. Neither 5% precision (LR) nor 90% precision at 56% recall (RF at 0.50) is the optimal operating point — that is what Phase 5 resolves.

Stratified 5-fold cross-validation confirmed RF's ranking advantage is not a test-set artifact: RF ROC-AUC 0.9735 ± 0.0201 vs. LR 0.9330 ± 0.0207 across all folds.



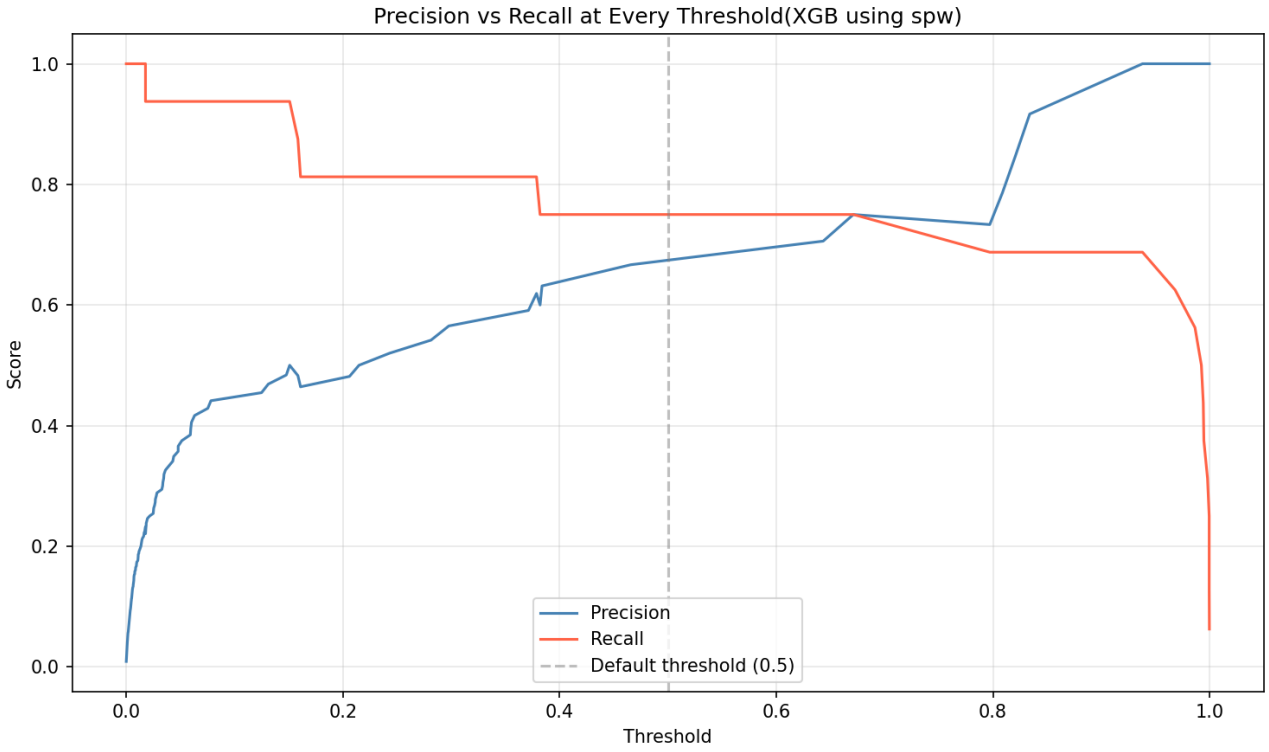
6. Final Model: XGBoost + SMOTE

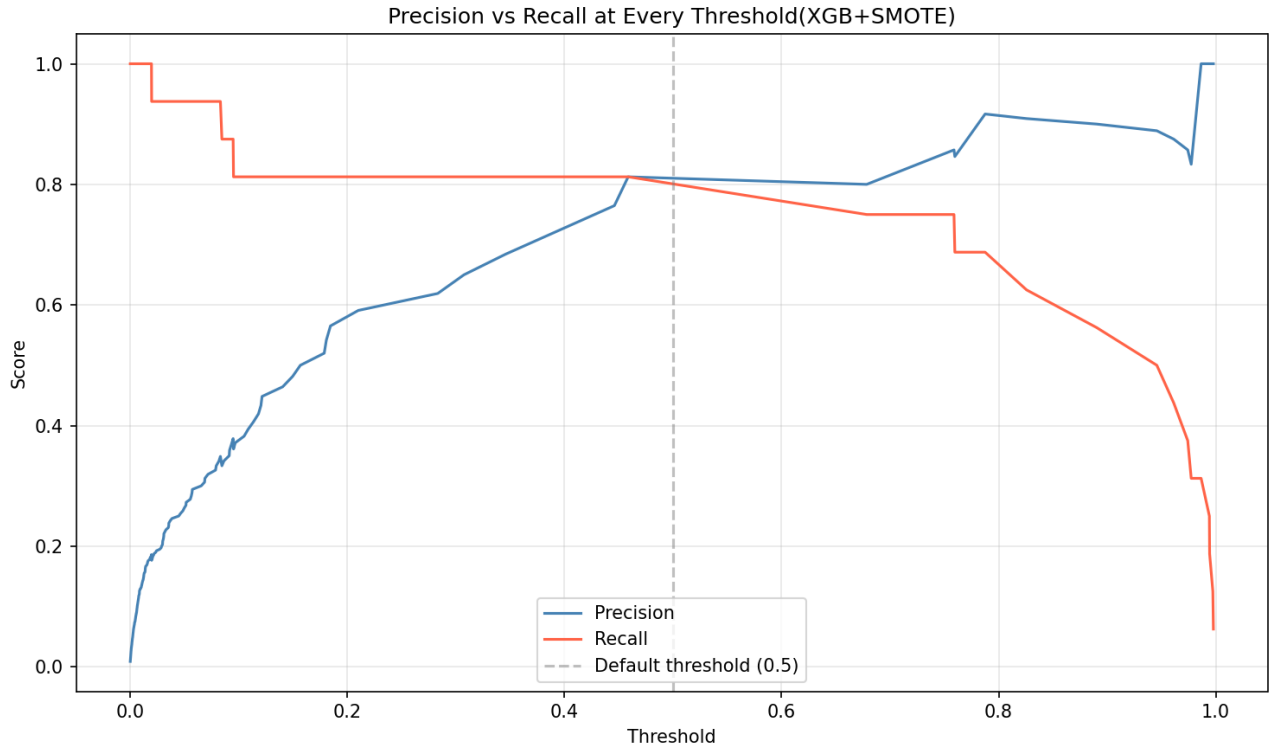
6.1 XGBoost over Random Forest

Random Forest trains trees independently and aggregates by majority vote; each tree is ignorant of the other trees' errors. XGBoost trains trees sequentially: each new tree corrects the residuals of the preceding ensemble, iteratively concentrating modelling capacity on the accounts currently misclassified. For tabular data with subtle, non-linear, multi-feature discriminative patterns — the defining characteristic of fraud detection — sequential boosting consistently outperforms parallel ensembling. This prior held empirically on this dataset, confirmed by OOF evaluation in Section 6.5.

6.2 Imbalance Handling: SMOTE vs. scale_pos_weight

Two strategies were compared. scale_pos_weight reweights the XGBoost loss function, penalising mule misclassifications more heavily (weight = n_negatives / n_positives ≈ 111). SMOTE generates synthetic minority samples by interpolating between real mule accounts and their k-nearest mule neighbours in feature space; at sampling_strategy=0.3, mules comprise 30% of the post-SMOTE training set (~2,100 synthetic + real mules against ~7,200 legitimate accounts). BorderlineSMOTE was evaluated and rejected: with only 65 training mules, neighbourhood-based boundary determination is insufficiently stable. Regular SMOTE with k_neighbors=5 is more reliable at this minority class size. The two mechanisms were not combined — SMOTE adjusts the training distribution and scale_pos_weight adjusts the loss function; using both simultaneously double-compensates for the same imbalance.





Configuration	Precision @ 0.5	Recall @ 0.5	ROC-AUC
XGB + scale_pos_weight	0.71	0.75	0.9967
XGB + SMOTE	0.80	0.75	0.9954

The ROC-AUC difference of 0.0013 across 16 test mules is within noise (one additional catch reverses the comparison). SMOTE's precision advantage at equivalent recall reflects better-calibrated probability estimates: rather than aggressively expanding the flagged region via loss reweighting, SMOTE exposes the model to greater variation in mule behaviour, producing probability scores that more meaningfully distinguish borderline cases. SMOTE was selected as the final imbalance strategy.

Implementation note: ImbPipeline (imbalanced-learn) is required when SMOTE is in the pipeline. sklearn's native Pipeline does not distinguish fit-time from predict-time for resampling steps — it would apply SMOTE to the test partition. ImbPipeline restricts SMOTE to .fit() calls only.

6.3 Hyperparameters

Parameter	Value	Justification
n_estimators	300	Matched to learning_rate=0.05; sufficient trees to converge at this dataset size without requiring early stopping
max_depth	4	Prevents overfitting to the 65-sample training minority class; depth 4 finds meaningful multi-feature patterns while being too shallow to memorize individual mule profiles

Parameter	Value	Justification
learning_rate	0.05	Conservative step size; prevents over-correction on small minority samples during sequential boosting
min_child_weight	3	XGBoost equivalent of min_samples_leaf: minimum samples in a node before a split is allowed. Enforces generalisation across multiple mule accounts rather than memorization of individual ones
subsample	0.8	Row-level randomisation per tree; reduces overfitting and creates diversity across sequential trees
colsample_bytree	0.8	Feature-level randomisation per tree; reduces overfitting
gamma	1	Minimum loss reduction required to make a split; prunes micro-splits driven by noise in minority class samples with limited real examples
scale_pos_weight	1	Neutral — SMOTE handles class imbalance; combining both mechanisms over-compensates

6.4 Threshold Tuning

The default threshold of 0.50 is a convention appropriate for balanced datasets. On imbalanced data, probability estimates skew toward the majority class; 0.50 is typically too conservative for the minority class. A systematic threshold sweep over the precision-recall curve identified the F1-maximising operating point:

Threshold	Precision	Recall	F1
0.40	0.765	0.8125	0.788
0.45	**0.8125**	**0.8125**	**0.8125**
0.50	0.800	0.7500	0.774

At threshold 0.45, precision and recall are exactly equal at 0.8125 — the crossover point and the F1 maximum by definition (the harmonic mean of P and R is maximised when both arguments are equal). The practical impact of this tuning is made concrete by Account #1460 in Section 7.3: threshold tuning from 0.50 to 0.45 is the direct and sole reason that account appears in the caught column.

6.5 OOF Evaluation

A single test set of 16 mules is too small for stable metric estimation. OOF evaluation — applying the 0.45 threshold to cross_val_predict(..., method='predict_proba') probabilities generated across all 65 training mules — provides a more reliable generalization estimate. Standard cross_val_score with a metric at the default threshold is insufficient; OOF collects probabilities first, then applies the chosen threshold once to the complete OOF probability set, honestly evaluating both model and threshold together.

Model	OOF Mule F1	OOF ROC-AUC
XGBoost + SMOTE	0.77	0.9649

Model	OOF Mule F1	OOF ROC-AUC
Random Forest	0.70	0.9538

XGBoost + SMOTE outperforms Random Forest on every OOF metric. The two models produced identical confusion matrices on the 16-mule test set (13 caught, 4 false alarms at their respective optimal thresholds), which would suggest equivalence. With 16 test mules, confusion matrices can only take a small number of distinct integer values; different models can produce the same counts by chance. OOF over 65 mules resolves the ambiguity — RF's apparent parity was a sampling artifact.

6.6 Test Set Confusion Matrix (threshold = 0.45)

	Predicted: Legitimate	Predicted: Mule
Actual: Legitimate	1,797 (True Negatives)	4 (False Alarms)
Actual: Mule	3 (Missed)	13 (Caught)

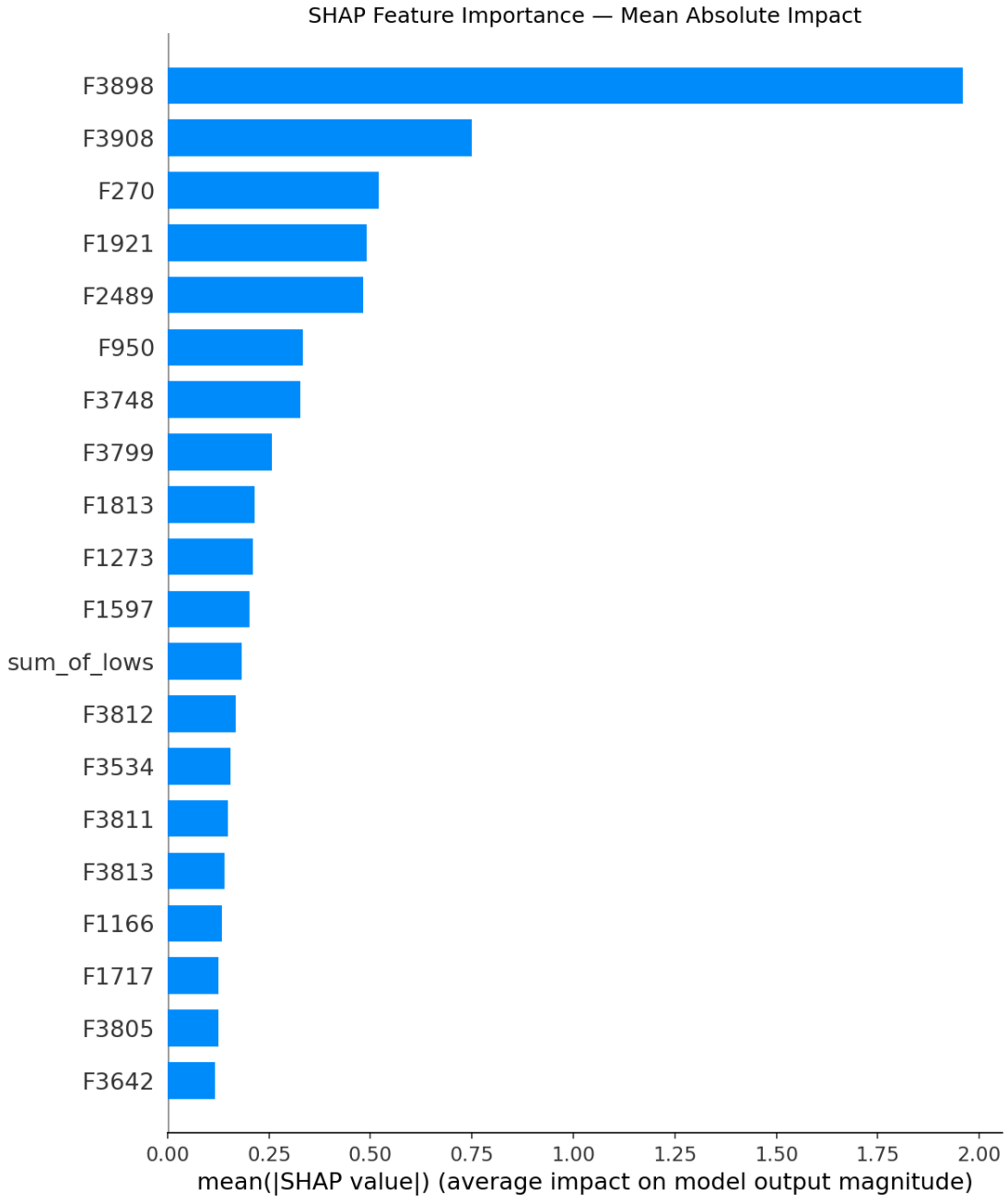
Operational summary: 13/16 mules caught (81.2% recall); 4 false alarms from 1,801 legitimate accounts (0.22%); investigation load of 1.3 alerts per confirmed mule — for every 13 accounts a fraud analyst investigates, approximately 10 are confirmed mules.

7. Explainability: SHAP Analysis

SHAP TreeExplainer is used for XGBoost — it is exact, not an approximation. Each prediction is decomposed into per-feature additive contributions: a positive SHAP value for a feature pushes the prediction toward mule; negative pushes toward legitimate. All contributions sum to the prediction minus the global baseline E[f(x)].

7.1 Global Feature Importance

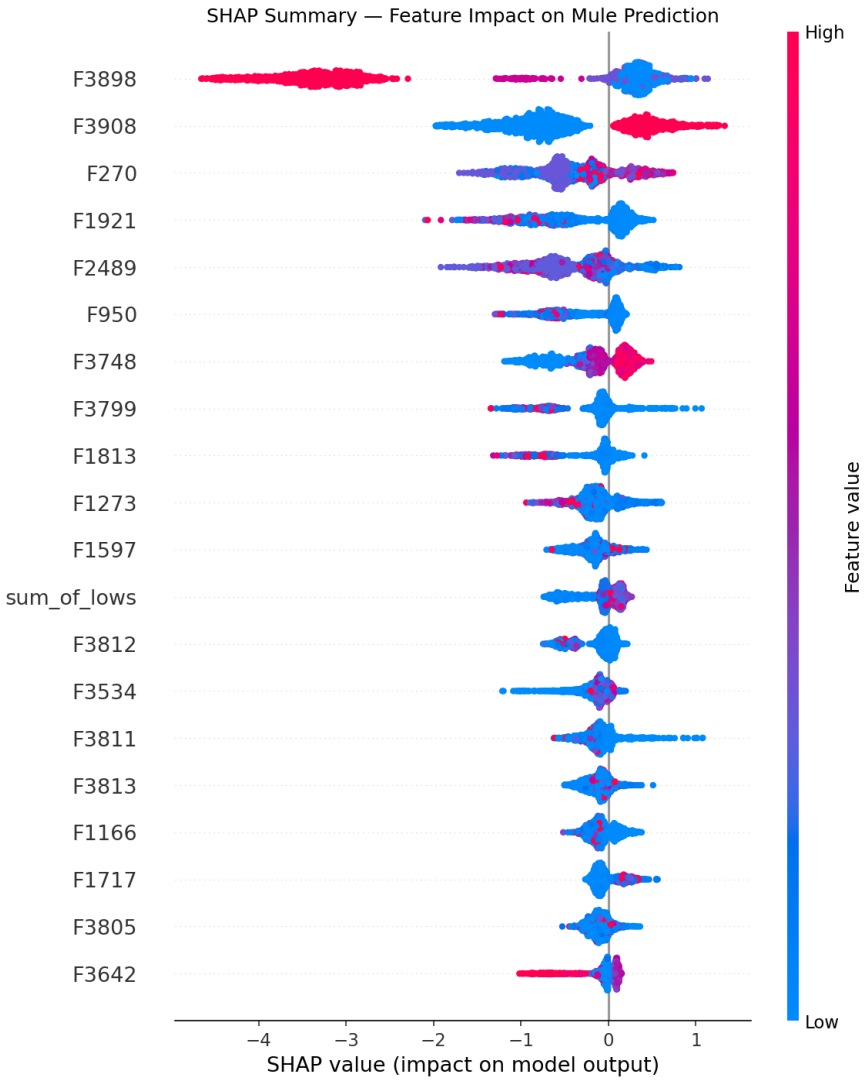
F3898 is the dominant predictor with mean absolute SHAP value approximately 2.0 — nearly 3x the second-ranked feature (F3908 at ~0.75). No other single feature approaches this dominance.



Rank	Feature	Mean |SHAP|	Interpretation
1	F3898	~2.0	Banking relationship depth: legitimate accounts accumulate depth (modal value 3); mule accounts are shallow (modal value 0 — created for immediate use, no relationship history)
2	F3908	~0.75	Flags mules through absence — low values carry the signal, not elevated ones
3	F270	~0.52	—
4	F1921	~0.50	Account activity volume; high values offer protection against mule classification (see Account #98)
5	F2489	~0.50	—
12	sum_of_lows	—	Engineered composite; position 12 confirms the feature engineering phase made a real contribution to the final model

The original 18 hint features (F2678, F2122, F2956, etc.) do not appear individually in the top 20 SHAP rankings. They are represented through sum_of_lows and superseded by features from the F38xx range identified during selection. The model found its primary signal in features not highlighted by the problem statement.

7.2 Beeswarm Interpretation

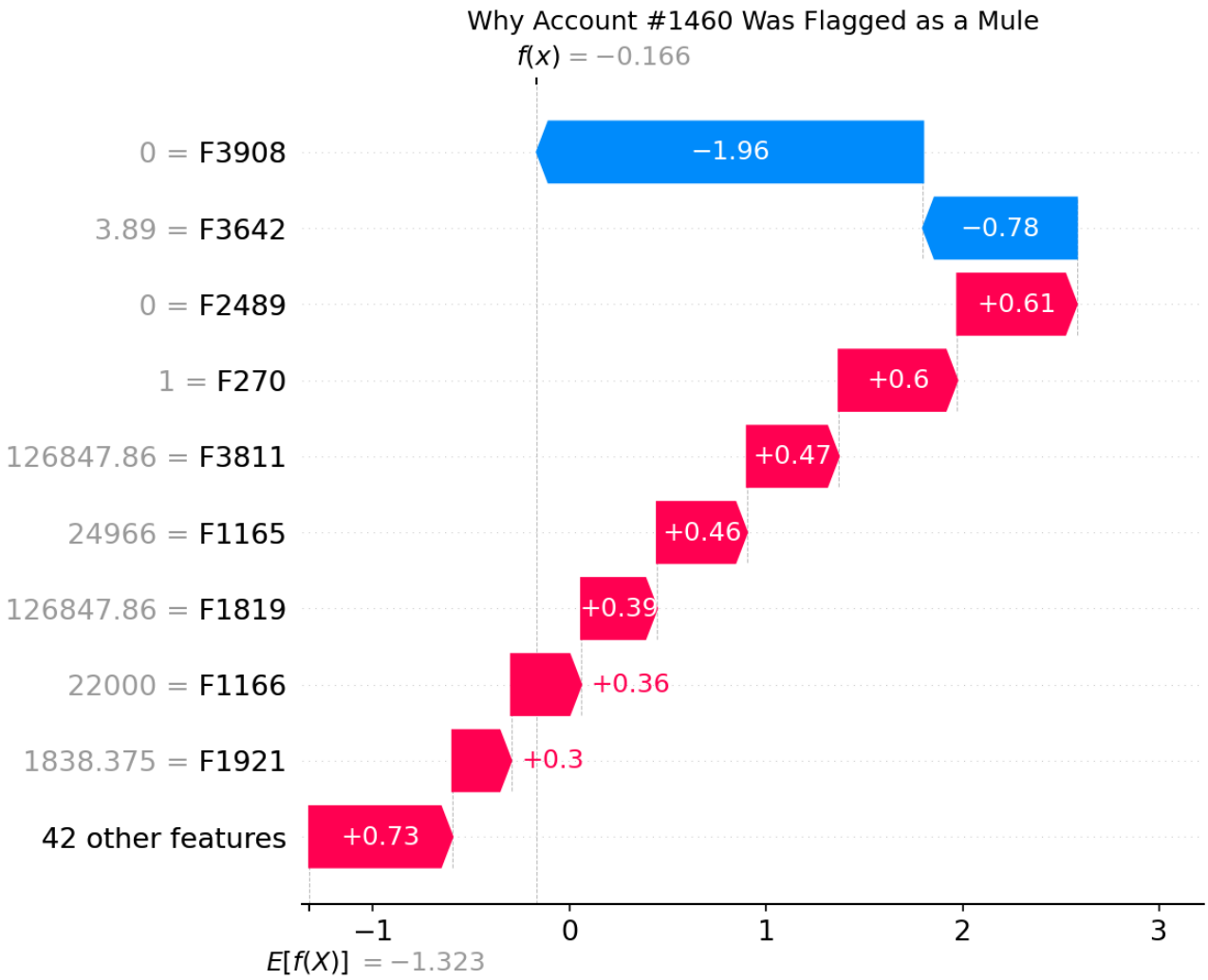


Feature	Beeswarm Pattern	Interpretation
F3898	High values (red) cluster at SHAP −3 to −4; low values (blue) sit near zero or slightly positive	Deep banking relationships strongly indicate legitimate accounts. Shallow profiles are the model's primary mule signal.
F3908	Low values (blue) spread rightward toward mule; high values (red) near zero	The feature discriminates through absence — low values are the signal.
sum_of_lows	Dots cluster on the left side (SHAP negative for high values)	Low composite activity score pushes toward mule — consistent with EDA findings that all five component features were lower for mules.

7.3 Individual Account Analysis

Account #1460 — Caught Mule (Margin: 0.009)

Final prediction: f(x) = −0.166 (log-odds). Converting to probability: sigmoid(−0.166) ≈ 0.459. Applied threshold: 0.45. Caught by a margin of 0.009.

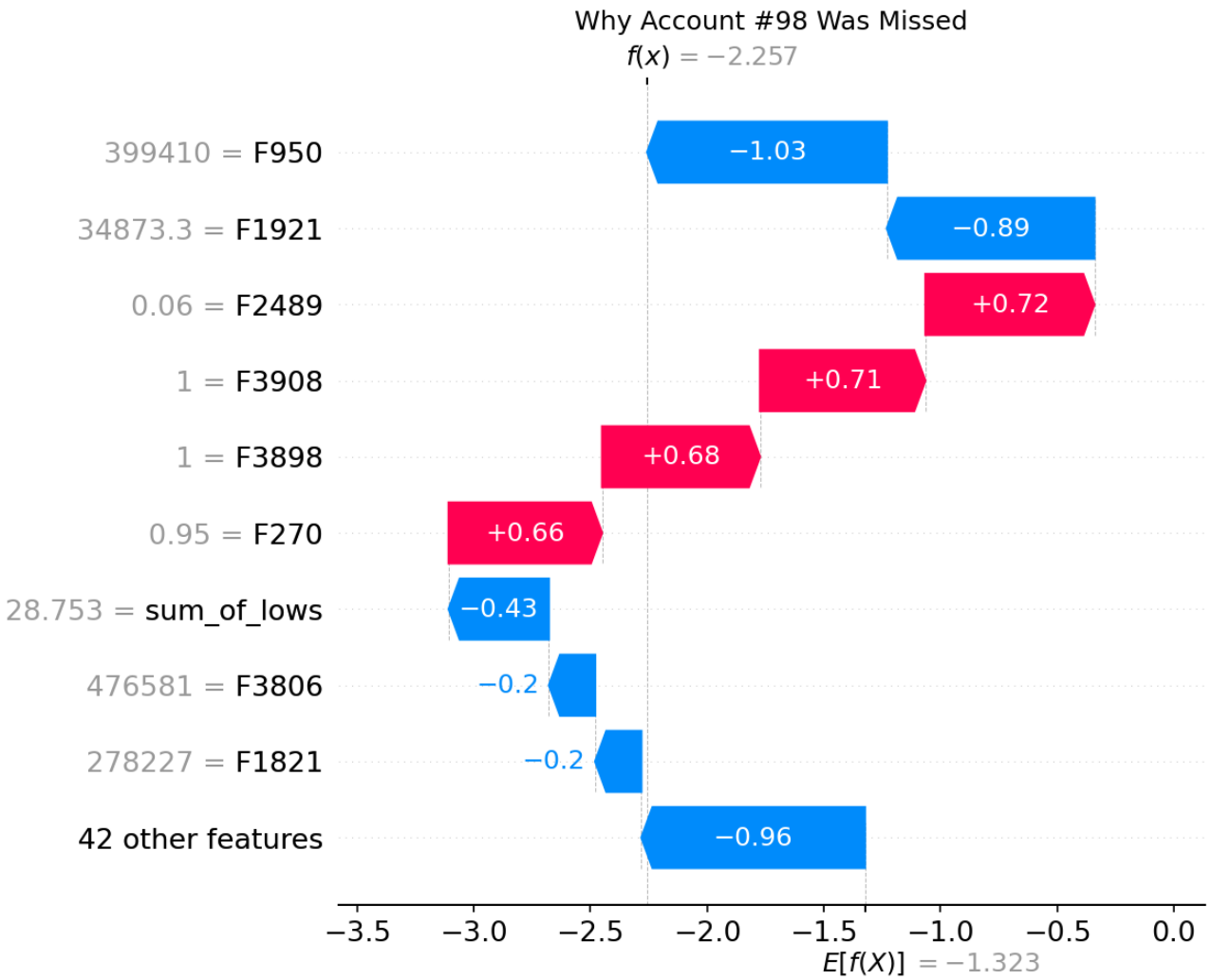


At the default threshold of 0.50, this account is a missed mule. Threshold tuning from 0.50 to 0.45 is the direct and sole cause of this catch — 1 of the 13 confirmed catches exists only because of deliberate threshold optimisation.

The SHAP waterfall reveals a significant internal contest: F3908 = 0 alone contributed −1.96 toward legitimate — a single feature nearly obscuring the account. Against it, a coalition of positive contributions: F2489=0 (+0.61), F270=1 (+0.60), F3811=126,847 (+0.47), F1165=24,966 (+0.46), F1819=126,847 (+0.39), F1166=22,000 (+0.36), F1921=1,838 (+0.30), plus 42 additional features combining for +0.73. The positive coalition collectively overcame the dominant negative pull, placing the final score 0.009 above threshold. This demonstrates SHAP's practical value: every flagged account can be explained to a compliance officer in terms of specific measurable behaviours, and every parameter choice — including the threshold — has traceable operational consequences.

Account #98 — Missed Mule

Final prediction: f(x) = −2.257. Baseline E[f(x)] = −1.323. The model moved further from mule than its prior — not toward it.



The miss is attributable to high transaction volume: F950 = 399,410 contributed −1.03 toward legitimate; F1921 = 34,873 contributed −0.89. Despite genuine mule-consistent signals — F3898=1 (+0.68), F3908=1 (+0.71), F2489=0.06 (+0.72), F270=0.95 (+0.66) — the transaction volume features and 42 additional features combined for −0.96, overwhelming the fraud signals entirely. This mule successfully disguised itself through activity levels indistinguishable from a legitimate high-volume account. The model had no discriminating information available to separate it. The miss reflects a genuine limitation of the available feature set, not a modeling error, and points directly to the graph-based extension described in Section 8.4.

8. Results & Conclusion

8.1 Final Performance Summary

Metric	Value
Model	XGBoost + SMOTE
Decision threshold	0.45
Test mules caught	13 / 16 (81.2% recall)
Test false alarm rate	0.22% (4 of 1,801 legitimate accounts)
Investigation load	1.3 alerts per confirmed mule
OOF mule precision	0.79
OOF mule recall	0.74
OOF mule F1	0.76
OOF ROC-AUC	0.9680

Test set metrics (0.8125 precision and recall) exceed OOF metrics (0.74–0.79) because the 16-mule test set is a mildly favorable sample. OOF evaluation applies the 0.45 threshold to cross_val_predict probabilities across all 65 training mules — every mule is evaluated by a model that never trained on it. This is the more conservative and reliable estimate of generalization performance. The gap is expected; a difference of one catch shifts test recall by 6.25 percentage points.

8.2 Operational Interpretation

An investigation load of 1.3 means that for every 13 accounts a fraud analyst reviews, approximately 10 are confirmed mules. At a false alarm rate of 0.22%, a bank with 1 million accounts would receive approximately 2,200 erroneous flags per monitoring cycle — operationally manageable against an 81.2% mule detection rate. The system materially outperforms rule-based alternatives (trivially gamed by threshold-aware adversaries), definitively outperforms the LR baseline (5% precision, unacceptable at any scale), and provides SHAP-level explainability for every flag — a prerequisite for compliance reporting in regulated banking environments.

8.3 Limitations

Limitation	Detail
Small test minority class	16 test mules produces wide confidence intervals on all point metrics; ±1 catch = ±6.25% recall. OOF over 65 mules is more reliable but still a small sample.
SMOTE diversity ceiling	~2,100 synthetic mules are derived from 65 originals at sampling_strategy=0.3. Synthetic diversity is fundamentally bounded by the real minority class size — the synthetics cannot capture mule behaviours not represented in the originals.

Limitation	Detail
High-volume mule accounts	Account #98 illustrates a systematic blind spot: mules disguised as high-activity legitimate accounts may be undetectable from account-level features alone. Graph-based structural features are needed to address this class of miss.
Feature interpretability	F3898's interpretation as banking relationship depth is inferred from its value distribution. No ground-truth feature dictionary was provided with the dataset.

8.4 Future Work

Direction	Rationale
Temporal validation	Hold out accounts by time rather than randomly to test whether the model generalises to future fraud patterns — a more realistic deployment scenario that avoids inadvertent look-ahead bias.
Graph-based features	Mule accounts form inter-account transfer networks. Node degree, betweenness centrality, and community membership can capture structural fraud patterns invisible to isolated account-level features — directly addressing the Account #98 failure mode.
Ensemble with graph detector	A graph-based component targeting high-volume accounts with suspicious network positions would complement the current model's strength on low-activity mules.
Probability calibration	Fit isotonic regression or Platt scaling to OOF probabilities to improve the semantic stability of the 0.45 threshold: calibrated probabilities would allow the threshold to carry a more literal interpretation as a posterior fraud probability across deployment contexts.